LAB SHEET 4

Full Stack Web Development — Node.js

Completed Answer Sheet — 12 Tasks with Output Screenshots

The answers and organization below follow the uploaded LAB SHEET 4 source. Example terminal values are illustrative outputs for demonstrating the expected result.

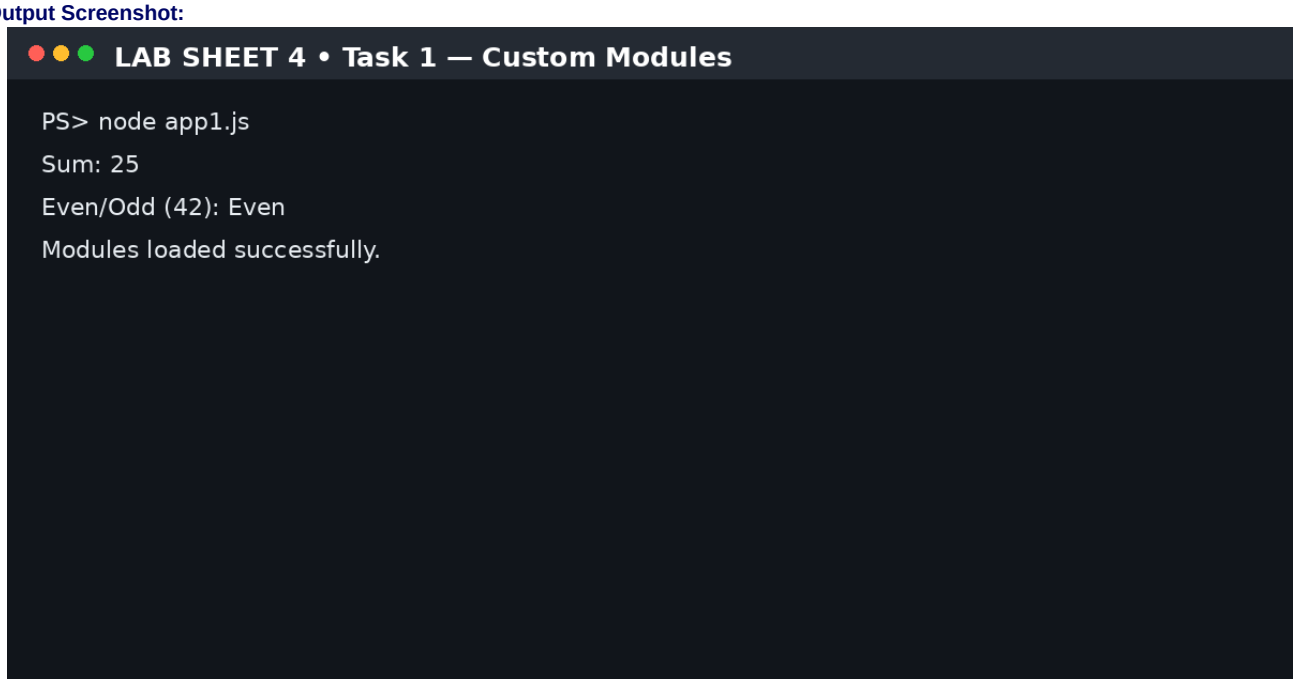
Part	Tasks	Topics
Node.js Fundamentals	1–6	Modules, File System, Directories, Path, OS, Events
Node.js Web / CLI	7–12	HTTP, Query Strings, Streams, npm, CLI Arguments, Environment Variables

Task 1: Custom Modules

Question: Create sum.js and even-odd.js, export one function from each, import both into app1.js, and call them with sample values.

Answer: sum.js exports a sum function and even-odd.js exports a checker. app1.js requires both modules and calls them with sample values.

Output Screenshot:



Task 2: File System — CRUD on a Text File

Question: Create student.txt, write name and roll number, append a course name, read and print the full content, then rename the file to profile.txt.

Answer: Use fs.writeFile to create student.txt, fs.appendFile to add the course, fs.readFile to display its content, and fs.rename to change it to profile.txt.

Output Screenshot:

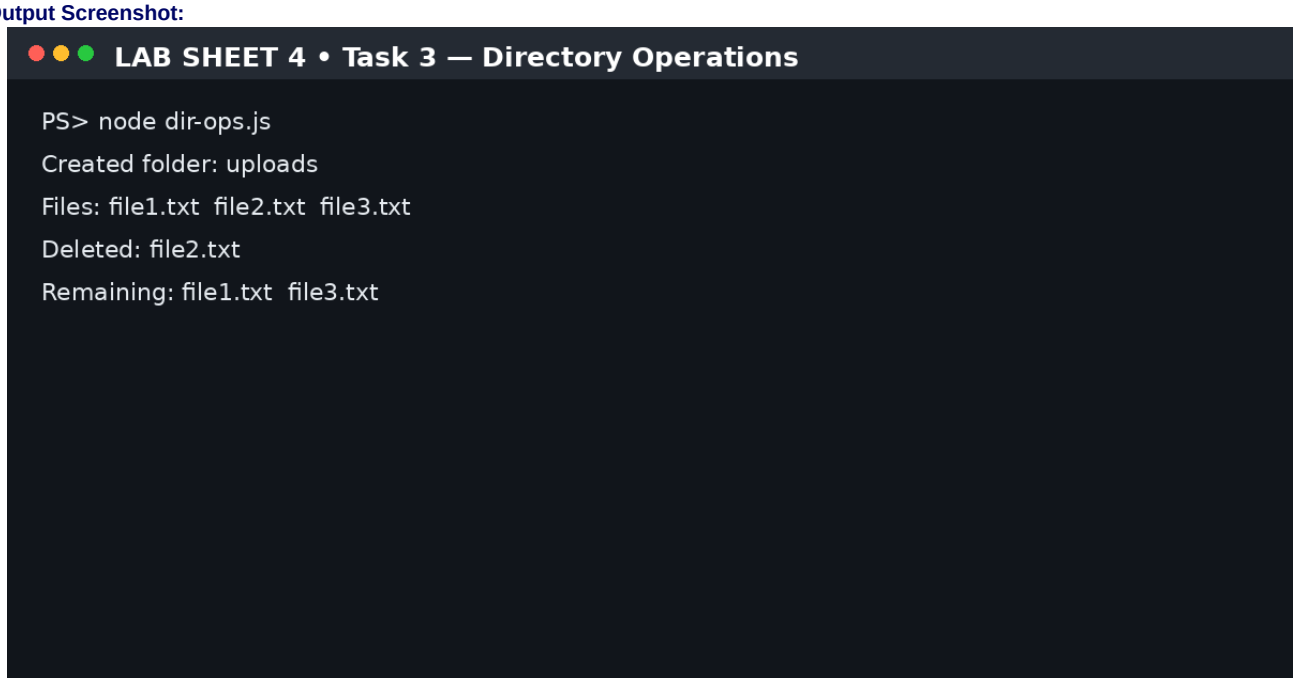


Task 3: Directory Operations

Question: Create uploads, create three empty files, list the files, then delete one file.

Answer: Use fs.mkdir for uploads, fs.writeFile three times for empty files, fs.readdir to list them, and fs.unlink to delete one.

Output Screenshot:

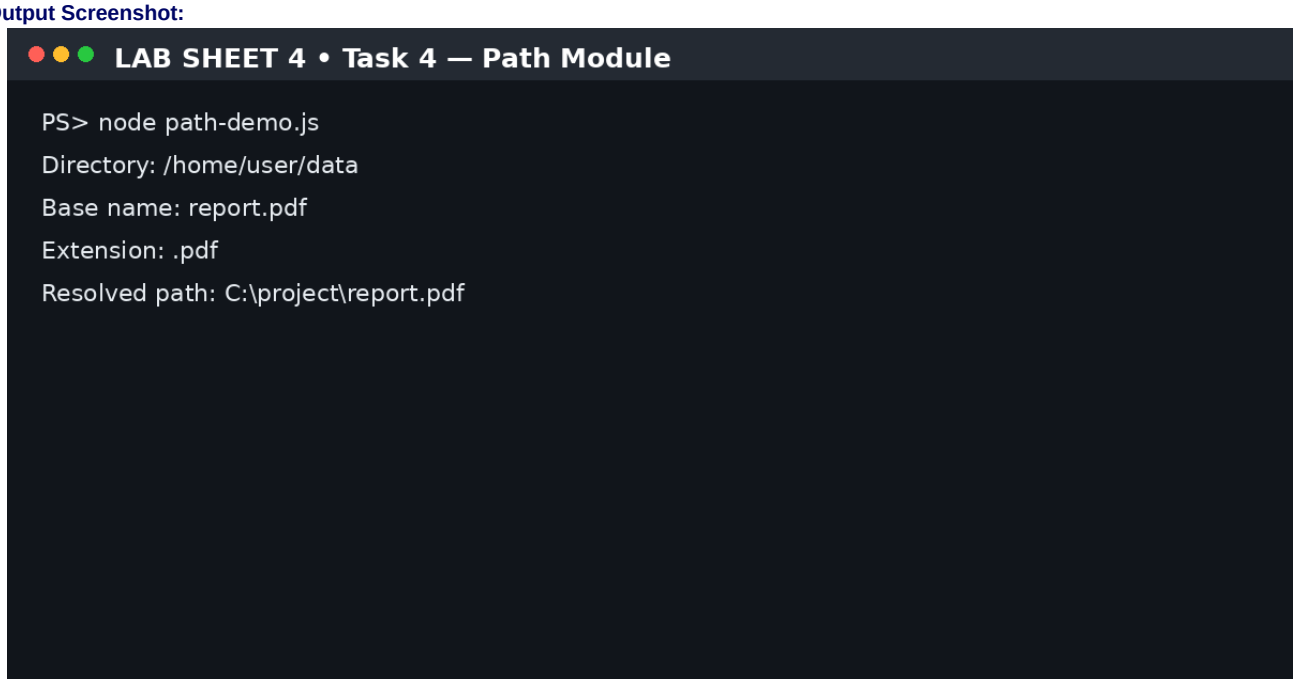


Task 4: Path Module

Question: Use a hardcoded path and print directory name, base name, extension, and an absolute path resolved from a relative one.

Answer: The path module exposes dirname, basename, extname and resolve. These functions separate path components and turn a relative path into an absolute path.

Output Screenshot:

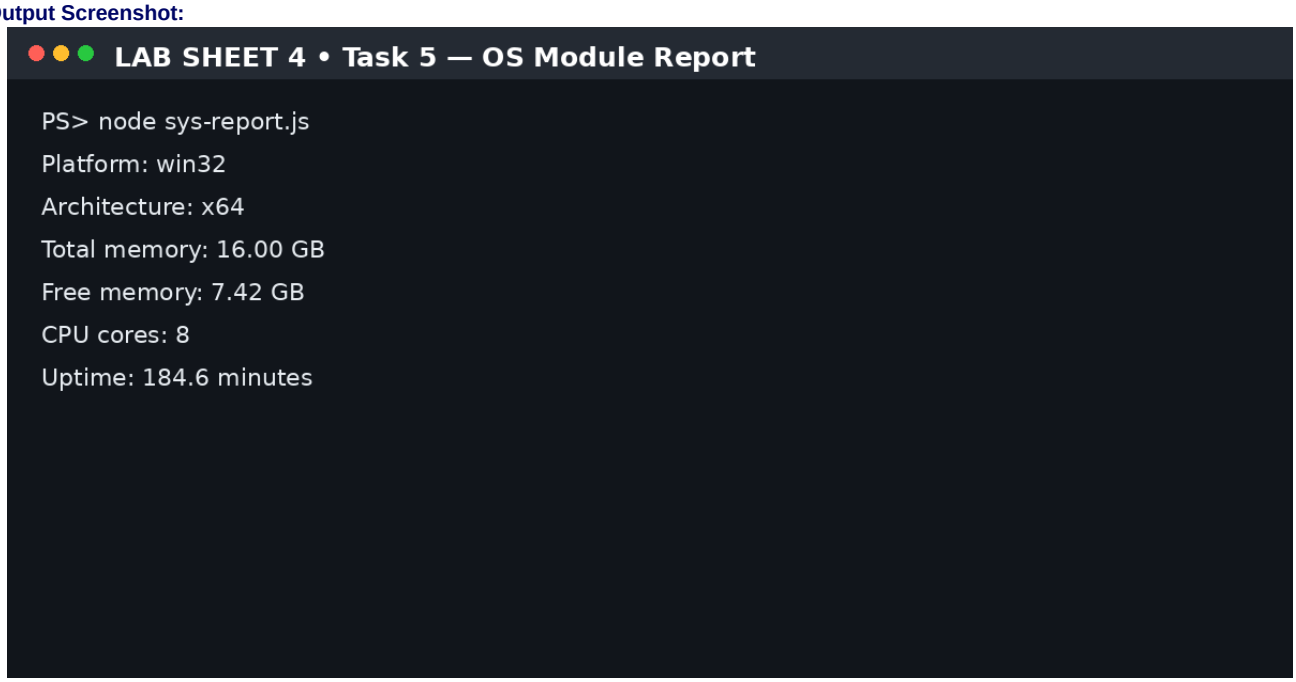


Task 5: OS Module Report

Question: Print platform, architecture, total/free memory, CPU core count, and uptime in minutes.

Answer: The OS module provides platform(), arch(), totalmem(), freemem(), cpus().length and uptime(). Divide uptime seconds by 60 to display minutes.

Output Screenshot:



Task 6: Events Module — Custom EventEmitter

Question: Define an EventEmitter for orderPlaced, pass id/item/amount, log a formatted receipt, and emit three orders.

Answer: Create an EventEmitter instance, register an orderPlaced listener, and emit three order objects. The listener formats each order as a receipt.

Output Screenshot:

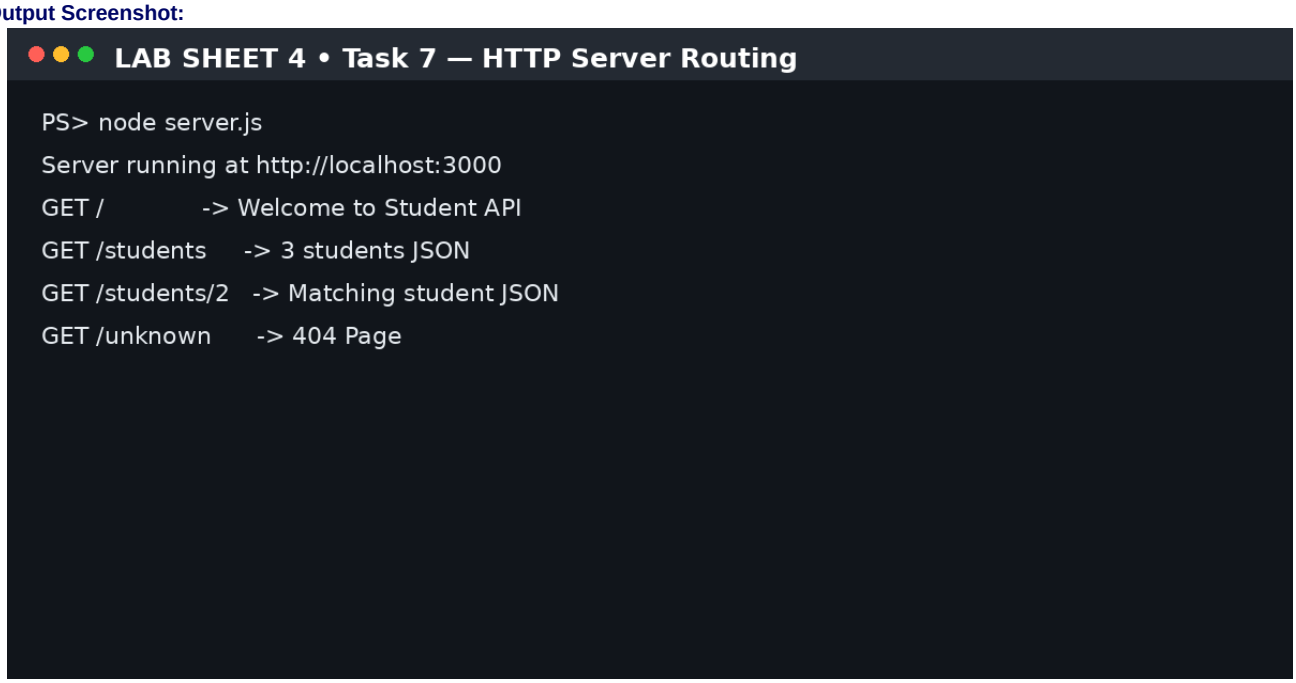


Task 7: HTTP Server with Manual Routing

Question: Create a framework-free server with /, /students, /students/:id and a catch-all 404 route.

Answer: Use Node's built-in http module and inspect req.url. Match /, /students, /students/, and send a 404 response for everything else.

Output Screenshot:

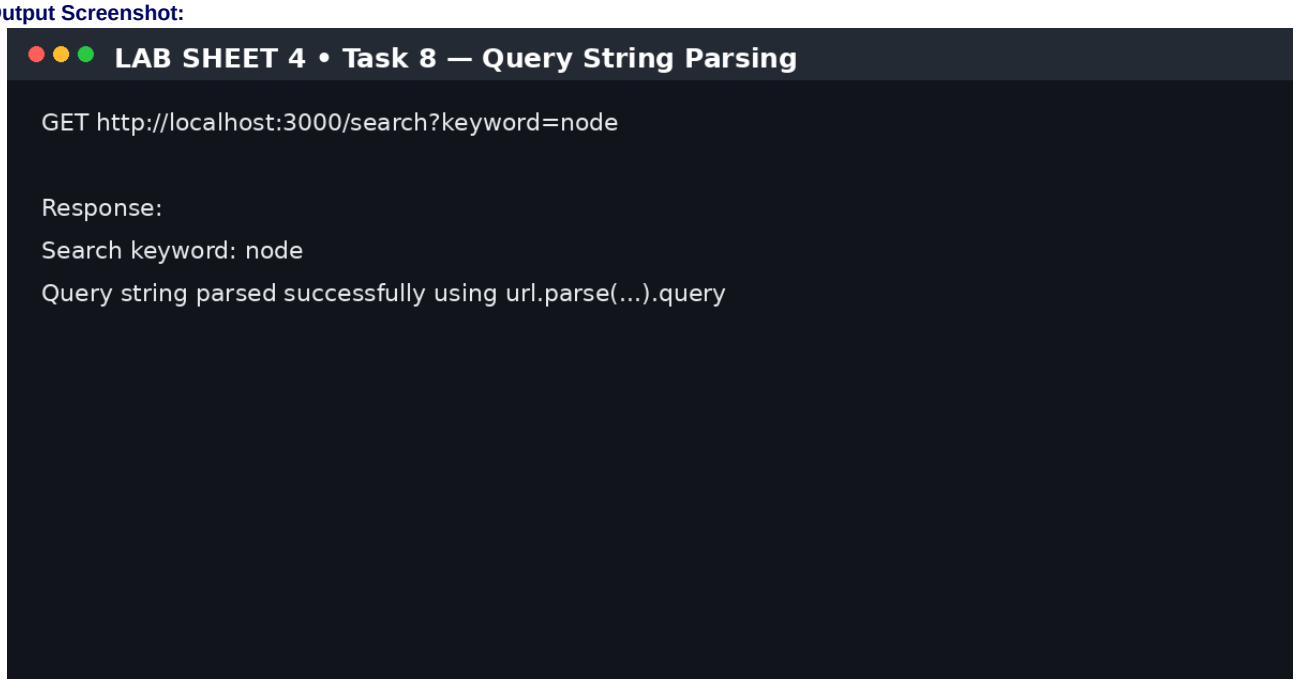


Task 8: Query String Parsing

Question: Create /search, read a keyword query parameter with the url module, and echo it back.

Answer: Parse the request URL with require('url').parse(req.url, true).query and read query.keyword. Echo that value in the response.

Output Screenshot:

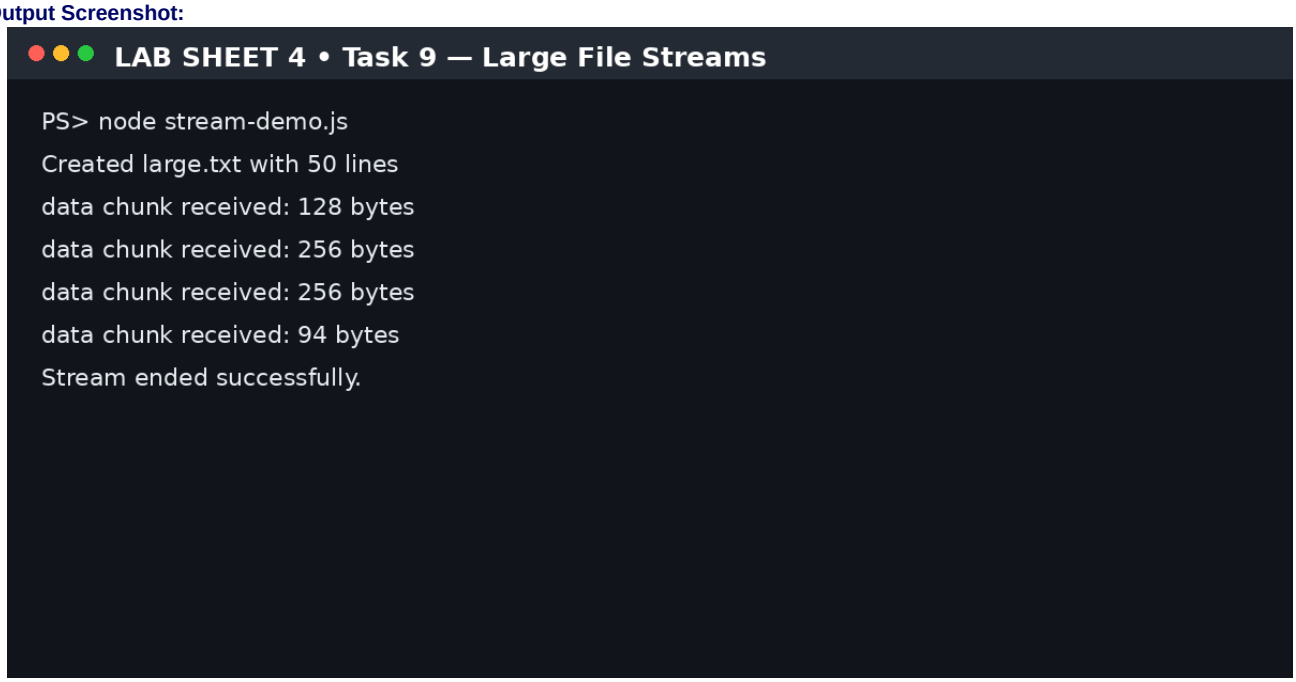


Task 9: Reading a Large File with Streams

Question: Create a text file with at least 50 lines, read it with createReadStream, and log chunk sizes on data/end.

Answer: Generate 50 lines with a loop, write them to a file, then create a read stream. The data event reports chunk.byteLength and the end event confirms completion.

Output Screenshot:

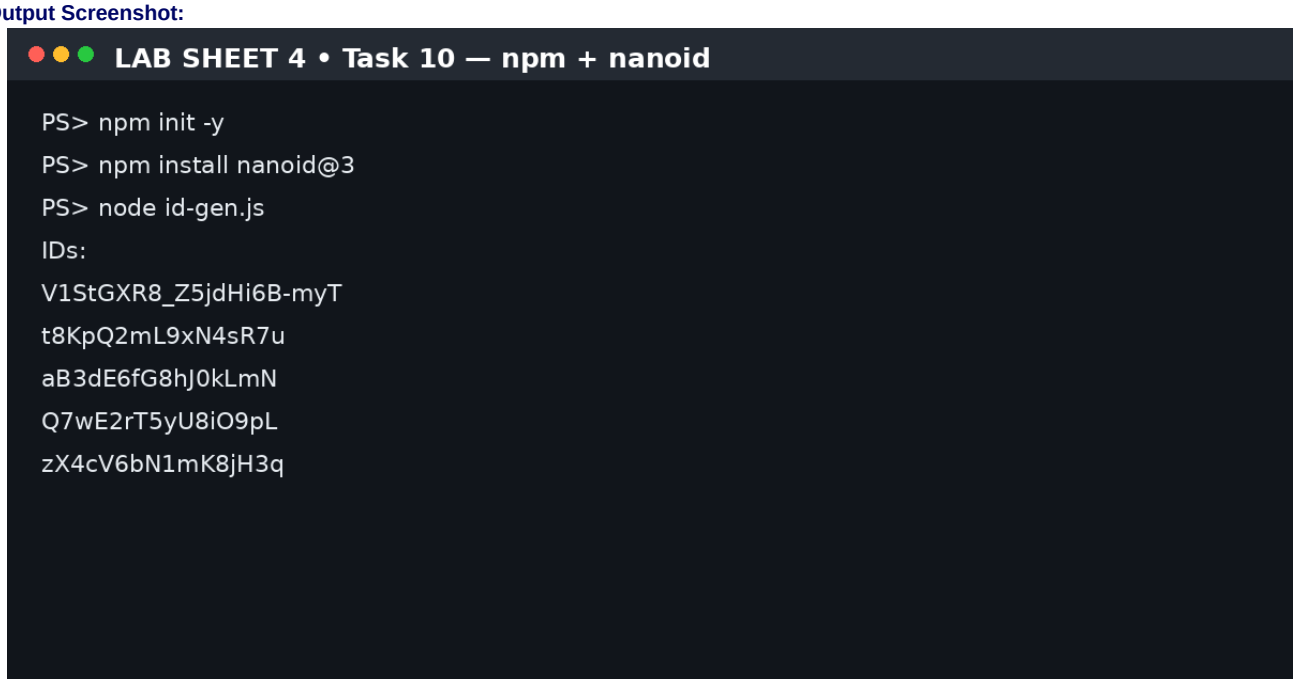


Task 10: npm and an External Package

Question: Run npm init -y, install nanoid@3, and generate five unique IDs.

Answer: Initialize the project with npm init -y, install nanoid@3, require nanoid and call it five times to print unique IDs.

Output Screenshot:

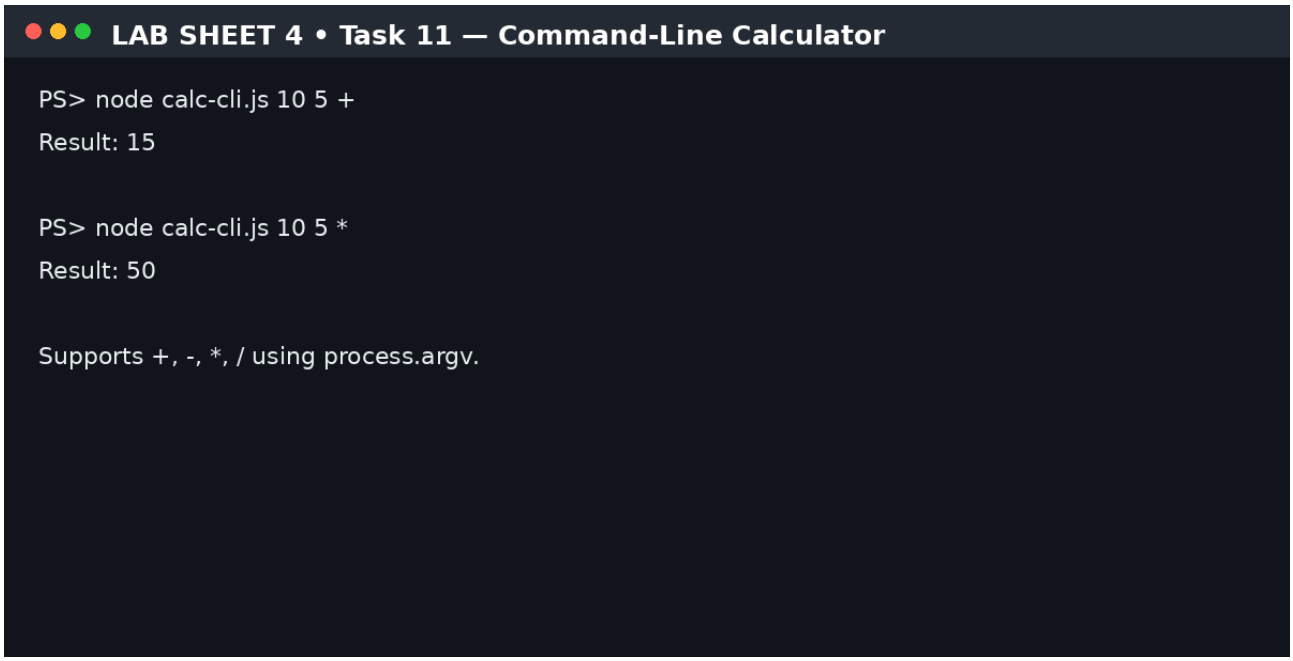


Task 11: Command-Line Arguments

Question: Accept two numbers and an operator using process.argv and support +, -, *, /.

Answer: Read process.argv[2], process.argv[3] and process.argv[4]. Convert the numbers and select the requested arithmetic operation.

Output Screenshot:

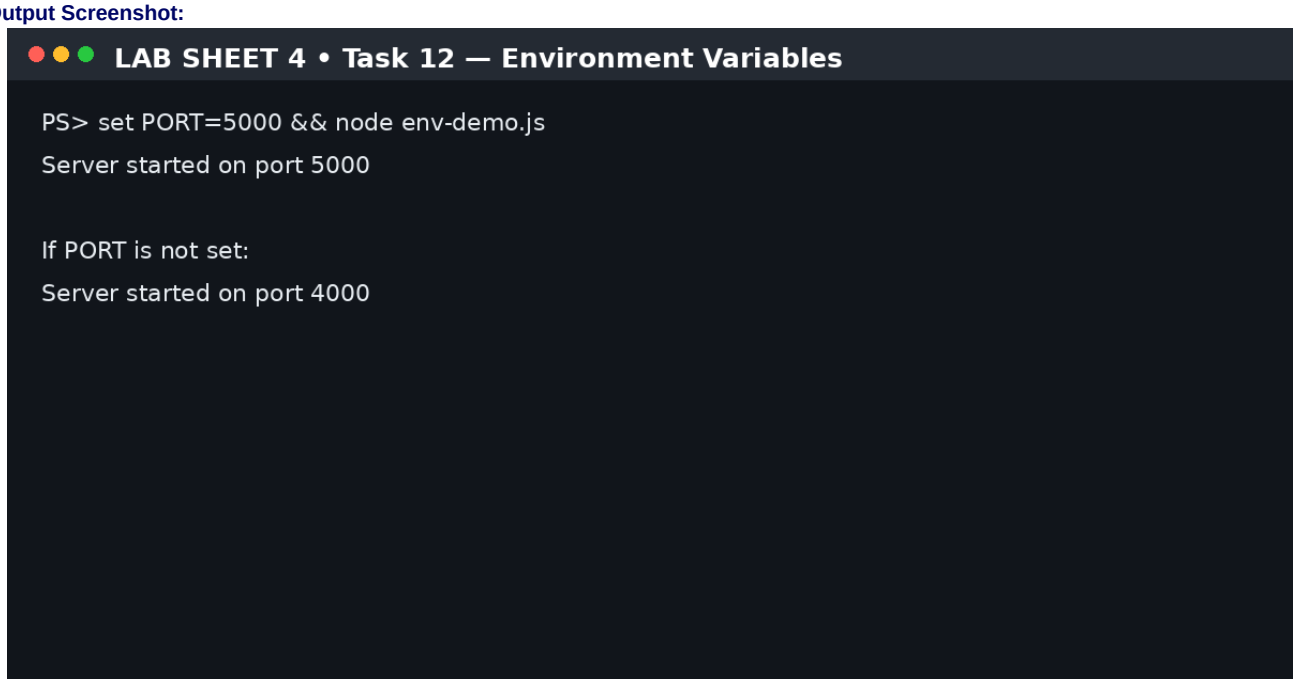


Task 12: Environment Variables

Question: Read PORT from process.env, default to 4000, start an HTTP server, and log the active port.

Answer: Read process.env.PORT and use 4000 when it is missing. Pass the resulting port to http.createServer(...).listen(port) and log it.

Output Screenshot:



LAB SHEET 4 — Completion Checklist

✓ 12 Node.js tasks answered

✓ Task-specific filenames and required Node.js APIs preserved

✓ Terminal/browser-style output included for every task

✓ HTTP routes and query-string behavior shown

✓ Streams, npm/nanoid, CLI arguments and environment variables demonstrated

Source: uploaded LAB SHEET 4 — Node.js document.